

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Database Scanning

Unit 5 • Ethical Hacking (10-151-160)



Learning Outcomes

By the end of this unit, you will be able to:



Explain databases and how they are queried

Identify common database platforms and describe how SQL retrieves and changes data.



Explain database permissions

Describe how privileges and roles decide what each account can reach.



Scan and extract from a target database

Run a scan, confirm an injection point, and extract information from it.



Analyze the results of scanning

Decide which findings are exploitable exposures versus noise.



Mitigate and detect database attacks

Recommend controls that stop exfiltration and detection that catches scans.

LESSON 1

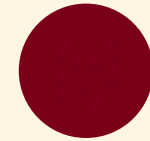
Databases and Queries



What Is a Database?

A structured set of data held in a computer, managed by software that answers requests for it.

- A **database** is a structured set of data — tables of rows and columns, or documents, or key-value pairs.
- A **database management system (DBMS)** stores the data and answers queries. You never touch the files directly.
- You ask for data by sending the DBMS a **query** in a language it understands.



Why databases are targeted

They hold **PII**, credentials, and payment data — the material that makes a breach worth money. The web server is often just the path to the database behind it.

Query Languages

Different kinds of database are queried in different ways:

Language	Database kind	Examples
SQL	Relational — tables of rows and columns	MySQL, SQL Server, PostgreSQL, Oracle, SQLite
LDAP	Directory services — a tree of objects	Active Directory
NoSQL	Non-relational — documents, key-value, graph	MongoDB and others

This unit focuses on SQL, the language of relational databases and the target of the most common database attack.

LESSON 2

Platforms and Vulnerabilities



Common Database Platforms

The platforms you are most likely to meet on an engagement:

Platform	What it is
MySQL	The most widely deployed open-source relational database; common behind web apps.
Microsoft SQL Server	Enterprise relational database tied closely to Windows and Active Directory.
PostgreSQL	Feature-rich open-source database; the fastest-growing of the major engines.
SQLite	A file-based database embedded in apps, phones, and browsers — not run as a server.
MongoDB	The most popular NoSQL document database; stores JSON-like records, not tables.

Every Platform Has Known Flaws

Once a scan gives you the product and version, look up its published vulnerabilities before going further.

- Each platform carries a running count of published **CVEs** — MySQL, SQL Server, PostgreSQL, SQLite, and MongoDB all have their own tallies.
- The exact number changes constantly; what matters is that it is **lookup-able** for a known product and version.
- Unpatched database software is a standing invitation — most breaches use a flaw that already had a fix.

Where to look

The **NVD** (National Vulnerability Database) lists every CVE for a product and version with severity scores. Vendor advisories name the fixing patch.

LESSON 3

SQL Injection



Reading a SQL Query

You do not need to write SQL fluently to assess a database, but you must be able to read it.

```
SELECT * FROM Users WHERE UserId = 105;
```

- `SELECT` retrieves rows; `INSERT`, `UPDATE`, and `DELETE` change them.
- The **WHERE clause** is the condition that decides which rows are affected — injection corrupts it.
- Everything after `--` is a **comment** the database ignores; attackers use it to cut off the rest of a query.

How SQL Injection Works

Placing crafted input into a field so it becomes part of the query the application runs.

- The flaw: the app builds a query by pasting input straight in — "...
`WHERE UserId = " + input.`
- The classic payload: `105 OR 1=1` makes the condition always true, returning every row.
- Stacked: `105; DROP TABLE Suppliers;` runs a second, destructive statement.
- Leads to reading, changing, or deleting data, dumping password hashes, and sometimes OS commands.



SECURITY FOCUS

SQL injection has ranked among the top web application risks on the OWASP Top 10 for years. It is not a virus and nothing gets installed — the attacker just types.

Why it matters: One unparameterized query can expose every row the app account can reach.

LIVE DEMO

Demo the OR 1=1 and stacked-statement examples on the W3Schools SQL injection page.

Stopping SQL Injection

The primary fix

- **Parameterized queries** — bind input as data, never as query text
- `105 OR 1=1` is then treated as a literal value, not code
- The single most effective control against SQLi

Defense in depth

- **Input validation** — reject input that is not the expected shape
- **Least-privilege accounts** — cap what a successful injection reaches
- **WAF** — flag injection patterns before they hit the app

Takeaway: Parameterize queries. Every other control reduces damage; parameterized queries prevent the flaw.

How Database Permissions Work

A database does not treat every connection the same — each account gets only the privileges it is granted.

- **Object privileges** — rights on specific tables or views, such as `SELECT` on one table but not another.
- **Administrative privileges** — server-wide rights like creating users or reading any table. An app account should never hold these.
- **Roles** — named bundles of privileges granted as a group, so permissions are managed once.
- **Least privilege** — an app limited to the tables it uses exposes far less through an injection flaw.

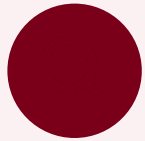
LESSON 4

Scanning Tools



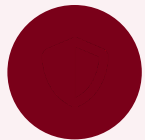
The Tools

A database scan has two sides — probing the app for injection, and checking the database itself.



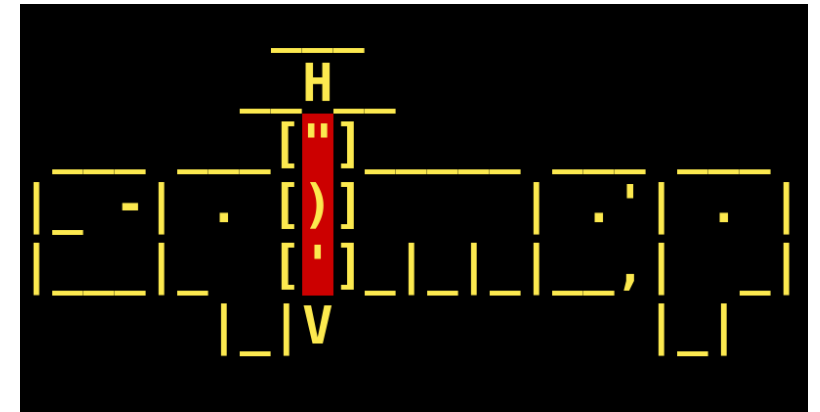
sqlmap

Open-source tool that automates detecting and exploiting SQL injection. Fingerprints the DBMS, enumerates databases, tables, and columns, and dumps their contents. Ships in Kali.



Scuba by Imperva (now Thales)

Free credentialed scanner that connects to a database and runs thousands of assessment tests for missing patches, weak passwords, and unsafe permissions — without running exploits.



Use either only against systems you are authorized to test.

LIVE DEMO

Run sqlmap against the authorized lab target and enumerate its databases.



DEFENSE

Mitigating and Detecting

Mitigating Data Exfiltration

Preventing a database breach is layered — no single control does it all.

- **Parameterized queries** — the most effective control against injection, and the reason "parameterize queries" is the SQLi takeaway.
- **Least-privilege accounts** — cap how much any single injection can exfiltrate.
- **Patch management** — apply the vendor fixes for the CVEs a scan turns up.
- **Encryption at rest & segmentation** — reduce the value of stolen data and keep the database off reachable paths.

Detecting Database Scans

A scan is noisy if anyone is watching — detection turns reconnaissance into an alert.

- **Query and error logging** — a flood of malformed queries and database errors is the signature of injection testing.
- **Web application firewall** — blocks or flags requests carrying injection patterns before they reach the app.
- **Database activity monitoring** — alerts on unusual row volumes, access to untouched tables, or logins from odd hosts.



SECURITY FOCUS

When a scan finds an exposure you cannot fix — a legacy app that needs a broad account, a vendor system you cannot patch — document it as an accepted risk with a compensating control.

Why it matters: An unresolvable finding becomes a tracked decision, not a silently ignored ticket.

Hands-On Lab: Scan and Extract

Kali lab · authorized target only · scan, extract, and defend

- Identify the database platform and version from scan output.
- Confirm an injection point and read why the payload changes the query.
- Use sqlmap to enumerate databases, tables, and columns.
- Extract a target table and identify what an attacker would do with it.
- Write the parameterized-query fix and the log signature that would have caught the scan.



Graded on the working extraction and the matching defense — a result mapped to a control, not a screenshot.

Key Takeaways



A database is where the valuable data lives — which is why it is the real target.



Least privilege decides how much an injection flaw can reach.



SQL injection abuses queries built from unsanitized input — parameterize queries.



sqlmap probes for injection; Scuba checks the database configuration.



Every finding ends in a control — mitigate, detect, or document as accepted risk.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 5 · Database Scanning

